When Wrappers Hide What a Language Model Knows: The Question Gets Through, the Answer Does Not

Shailesh Rana
Independent Researcher

Abstract

Language models now often read questions inside tool results, web pages, logs and emails. We test whether such a wrapper makes a model lose answers it knows, and where in the network the answer is lost. We asked four open models (Qwen3-4B, Qwen3-8B, OLMo-3-7B and Gemma-4-12B) 1,606 short questions about facts, arithmetic and translation, kept for each model only those it answers correctly when asked plainly, and placed each inside five kinds of wrapper at three lengths. Tool results, web pages and email threads lose 6%–18% of known answers, depending on the model, against a noise floor of 0.3%–1.2%; plain prose of the same length loses about as many, and server logs lose 6%–39%. Logs mostly change the task, so that the reply describes the log, whatever the model knows. The other wrappers mostly turn weakly held answers into wrong answers, or into claims that the text holds no answer. To find where the answer is lost, we copied internal states from the plain prompt into failed wrapped prompts. With another question's states at the question's tokens, the model gives that other question's answer in up to 51%–83% of failures, as often as when the other question is asked in the wrapper. Restoring the failed question's own states helps in at most 47%. Restoring the last position helps only from 0.59–0.69 of the network's depth, where the question's content reaches it. The model reads the question through the wrapper; the answer is lost later, at the position where the reply begins.

1 Introduction

Ask Qwen3-4B "Name the capital city of Mongolia." and it answers "Ulaanbaatar". Put the same line inside 2,000 tokens of a server log, followed by the same request to answer the question, and it replies "The logs show that multiple jobs ... are failing with a TimeoutError" (Figure 1). The model has not forgotten the capital of Mongolia. Something in the wrapper keeps it from giving the answer.

Language models now often meet questions inside such wrappers: the output of a tool, a web page, an email thread, a log file. Changing only the format of a prompt can move accuracy by tens of points [6, 18], and much of that change comes from replies that break the requested format [13]. Two questions remain open:

1. How often does a wrapper make a model lose an answer it knows, rather than only change how the answer is written?
2. Where in the network is the answer lost?

We asked four recent open models (Qwen3-4B, Qwen3-8B, OLMo-3-7B and Gemma-4-12B) 1,606 short questions about facts, arithmetic and translation. For each model we kept only the questions it answers correctly when asked plainly. We placed each question inside five kinds of text, at three lengths, and counted how often the known answer was missing from the reply. To find where it goes missing, we copied internal states from the plain prompt into failed wrapped prompts. We find the following.

- *Wrappers hide known answers* (Section 3.1). Tool results, web pages and email threads hide 6%–18% of known answers, depending on the model, against a noise floor of 0.3%–1.2%. Plain prose of the same length hides about as many. Server logs hide the most in two of the four models.
- *Logs change the task; other wrappers change the answer* (Section 3.2). Most answers lost to logs are replaced by a description of the log. Answers lost to other wrappers become wrong answers, or claims that the text does not contain the answer.
- *Wrong answers replace weakly held ones* (Section 3.3). Wrong answers and claims that the answer is not there almost disappear when the model gives the answer a probability above 0.999 without a wrapper. Replies about the log do not: they replace firm and weak answers alike.
- *The model reads the question; the answer is lost later* (Section 3.4). Swapping another question's internal states into a failed prompt makes the model give that other question's answer, as often as when the other question is asked in the wrapper. Restoring the failed question's own states helps much less. Restoring the last position does help, but only once the question's content has reached it, at 0.59–0.69 of the network's depth.

2 Setup

Appendix A gives the details needed to reproduce the results.

2.1 Questions the model knows

A wrapper can only hide an answer that the model has. We therefore use short questions with one correct answer and keep,

for each model, only those it answers correctly without a wrapper. We built 1,606 such questions in three families from public datasets:

- *Facts* (839): the capital of a country [21]; the country of a landmark, the city of a company's headquarters, the constellation of a star and the real name of a superhero [8].
- *Arithmetic* (450): sums, differences and products of two numbers, such as "What is 64 plus 22?".
- *Translation* (317): an English word in French, German or Russian [25].

Each item is worded either as a question ("What is the capital city of Mongolia?") or as an instruction ("Name the capital city of Mongolia."), at random.

In the *plain prompt*, the item follows "Question:" and is followed by one instruction, "Answer the question above. Reply with only the answer, in plain text." The model's reply is then started with "Answer:". An item is *known* to a model if the model's greedy reply to the plain prompt is correct and the model gives the full answer a probability of at least 0.5. Each model knows 71%–80% of the items. We split items by subject, number pair or English word into a development set (30%), used only to choose settings, and a test set (70%), which gives every reported number.

2.2 Wrappers

A wrapper places the plain prompt's question line inside text of the kind an assistant reads through tools. There are four interface formats and one control:

- *Tool result*: the JSON output of an inbox search; the question is the text of one message.
- *Web page*: the HTML of a forum thread; the question is one post.
- *Server log*: timestamped log lines, with occasional errors and stack traces; the question is the payload of one request.
- *Email thread*: a chain of replies; the question is the body of one email.
- *Plain prose* (control): paragraphs of ordinary sentences without any markup.

The surrounding text is bland office news, such as "The heating in the north wing was serviced at the end of the day." It never contains an item's answer and asks no questions. Each format has 16 such texts, written by a seeded generator.

After the question come about 500 more tokens of the same wrapper and then the closing instruction of the plain prompt. We vary the amount of text *before* the question in three steps, so that the whole prompt has about 700, 2,000 or 16,000 tokens. Everything from the question onward is the same at every length. Figure 1 shows part of one wrapper. We fixed in advance that the wrappers must lose clearly more answers than the noise floor before we study where the loss happens; a first layout, with only about 64 tokens after the question, lost too few in Qwen3-4B, so we used this one (Appendix A).

To measure how often an answer is lost for reasons unrelated to the wrapper, two *null wrappers* add one neutral sentence to the plain prompt, before the question ("Something I have been wondering about.") or after it ("Thanks in advance."). Their loss rate is the *noise floor*.

```
==> /var/log/app/mailer.log <==
  ...
09:38:05 INFO  [scheduler] cache hit ttl=300
09:38:08 WARN  [mailer] /v1/files/upload 200
09:38:11 INFO  [gateway] /v1/messages user_message:
  Question: Name the capital city of Mongolia.
09:38:14 WARN  [scheduler] /v1/contacts 404
09:38:19 ERROR [auth] Unhandled exception in job
  sync_contacts
Traceback (most recent call last):
  File "/srv/app/clients/crm.py", line 117
    raise TimeoutError("upstream timed out")
TimeoutError: upstream timed out after 30s
  ...

Answer the question above. Reply with only
the answer, in plain text.
```

Figure 1: A question inside a server-log wrapper. Part of a 2,000-token wrapper, with most lines cut and the rest shortened. Without the wrapper, Qwen3-4B answers "Ulaanbaatar". With it, Qwen3-4B replies "The logs show that multiple jobs—notify_contacts, export_contacts, and sync_contacts—are failing with a TimeoutError".

2.3 Kept answers

For every known item and every condition, the model replies greedily with up to 24 tokens, stopping at the first line break. The answer is *kept* if the known answer appears in the reply as a whole word, in any format; otherwise it is *lost*. Kept answers include replies such as "The capital city of Mongolia is Ulaanbaatar.", which ignore the request for only the answer. We count these as kept because the model still reached the answer. A program that expects only the answer would see more failures: averaged over models, only 49%–86% of wrapped replies are exactly the answer, depending on the wrapper (Appendix B).

2.4 Models

We study four recent open instruction-tuned models from three developers: Qwen3-4B-Instruct-2507 and Qwen3-8B [17], OLMo-3-7B-Instruct [20] and Gemma-4-12B-it [3]. Each model uses its own chat template, with reasoning turned off, and runs in 16-bit precision.

3 Results

3.1 Wrappers hide answers the model knows

Adding one neutral sentence to the plain prompt loses 0.3%–1.2% of known answers, depending on the model. This noise floor is measured on the short plain prompt; plain prose is the control of the same length. Wrappers lose many more answers (Figure 2). Averaged over the three lengths:

- tool results, web pages and email threads lose 6%–18% of known answers, depending on the model;
- plain prose loses 8%–18%;

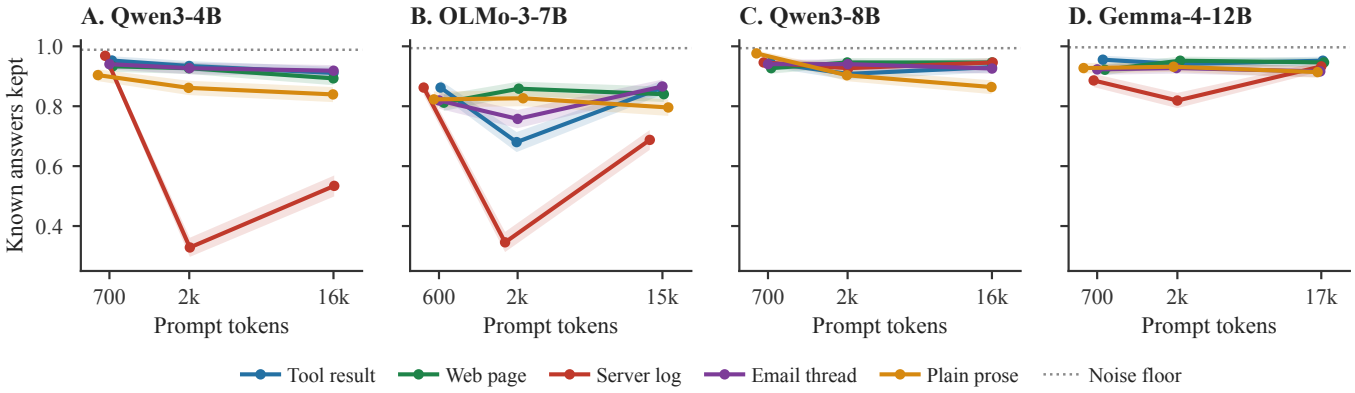


Figure 2: Known answers kept inside wrappers. Share of known test items whose reply still contains the known answer, for each wrapper and prompt length (log scale; lengths differ slightly between tokenizers). The dotted line is the noise floor: the share kept when one neutral sentence is added to the plain prompt. Bands are 95% intervals.

- server logs lose 6%–39%, and up to 67% at one length.

On average, the interface formats hide about as many answers as plain prose of the same length, so the loss does not need markup. Logs are the exception: they are the most damaging wrapper for Qwen3-4B and OLMo-3-7B, while Qwen3-8B and Gemma-4-12B lose at most five points more to logs than to any other wrapper. Length has no consistent effect: in the models hurt most by logs, logs lose most at the middle length, and the other wrappers change by a few points from one length to the next.

How much wrapper sits between the question and the closing instruction matters for logs. For Qwen3-4B we also ran every wrapper with about 64 instead of about 512 tokens of wrapper after the question. Logs then lost 19% of known answers instead of 39%. The other four wrappers, averaged, lost 9% in both layouts (Appendix C).

3.2 What replaces a lost answer

We sorted every lost answer into one of three kinds by the first line of the reply (Appendix D gives the rules):

- *about the wrapper*: the reply describes the wrapper's content, such as "The logs show that multiple jobs are failing";
- *not there*: the reply says the answer cannot be given, such as "The provided text does not contain information about the headquarters location of Taito Corporation";
- *wrong answer*: the reply gives another answer, such as "Sydney" for the capital of Australia.

Table 1 shows the result over all four models. Logs change the task: 64% of the answers they hide are replaced by a description of the log. The other wrappers rarely do this. Their lost answers become wrong answers or claims that the text holds no answer. Plain prose draws such claims most often (73% of its losses), although every question asks for general knowledge that no text needs to supply.

Wrapper	About the wrapper	Not there	Wrong answer	n
Tool result	2%	21%	77%	980
Web page	3%	49%	48%	923
Server log	64%	8%	28%	2,344
Email thread	1%	46%	53%	1,010
Plain prose	0%	73%	27%	1,207

Table 1: What replaces the lost answers, as a share of all lost answers for each wrapper, over the four models and three lengths. n is the number of lost answers.

3.3 Wrong answers replace weakly held ones

Which known answers are lost? We grouped known items by the probability the model gives the full answer in the plain prompt and counted what each wrapped prompt produced (Figure 3). The two kinds of failure behave differently.

- *Wrong answers and "not there" claims hit weakly held answers.* Outside logs, among answers the model gives with probability 0.5–0.8, wrappers turn 26% into wrong answers and 15% into claims that the answer is not there. Above 0.999 these fall to 1.2% and 1.3%. A neutral sentence also loses more of the weakest answers (6%, against at most 1.3% in the other groups), but far fewer than the wrappers.
- *Replies about the wrapper hit firm answers too.* Under logs, the share of replies that describe the log does not fall as the model grows more sure; it is 14% for the weakest group and 15% for the firmest.

So a wrapper that changes the task overrides what the model knows, however firmly. The other failures mostly take answers the model holds weakly.

3.4 The model reads the question; the answer is lost later

Where in the network is the answer lost? One possibility is that the wrapper changes how the model reads the question, so that the question's tokens no longer carry what the model needs. Another is that the question is read correctly, and the

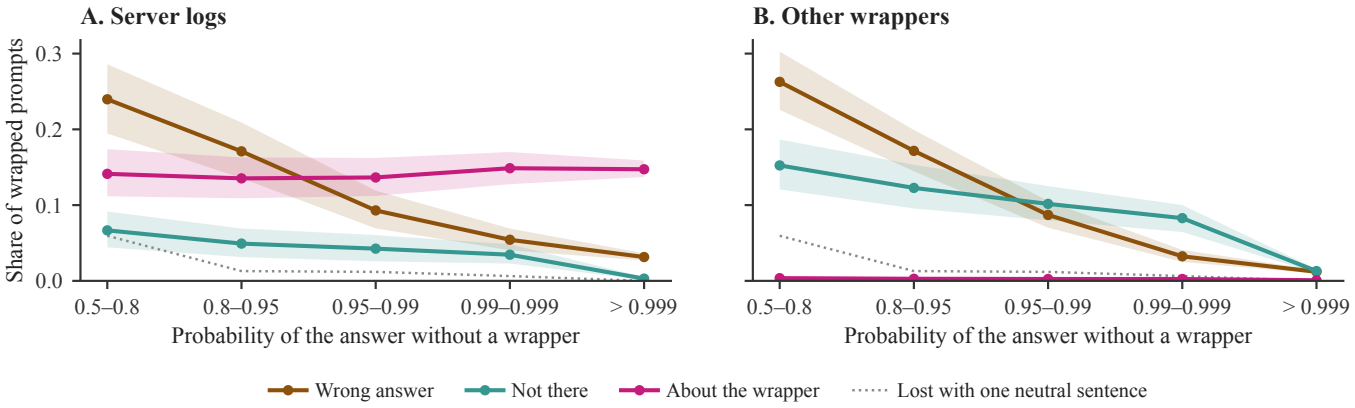


Figure 3: What wrapped prompts produce, by how sure the model is of the answer without a wrapper. Known test items are grouped by the probability each model gives the full answer in the plain prompt (3,150, 4,065, 5,055, 7,095, 31,935 wrapped prompts per group, over all four models, wrappers and lengths). Each coloured line is the share of wrapped prompts that lose the answer in one way; the remaining prompts keep it. The dotted line is the share lost when one neutral sentence is added instead. Bands are 95% intervals.

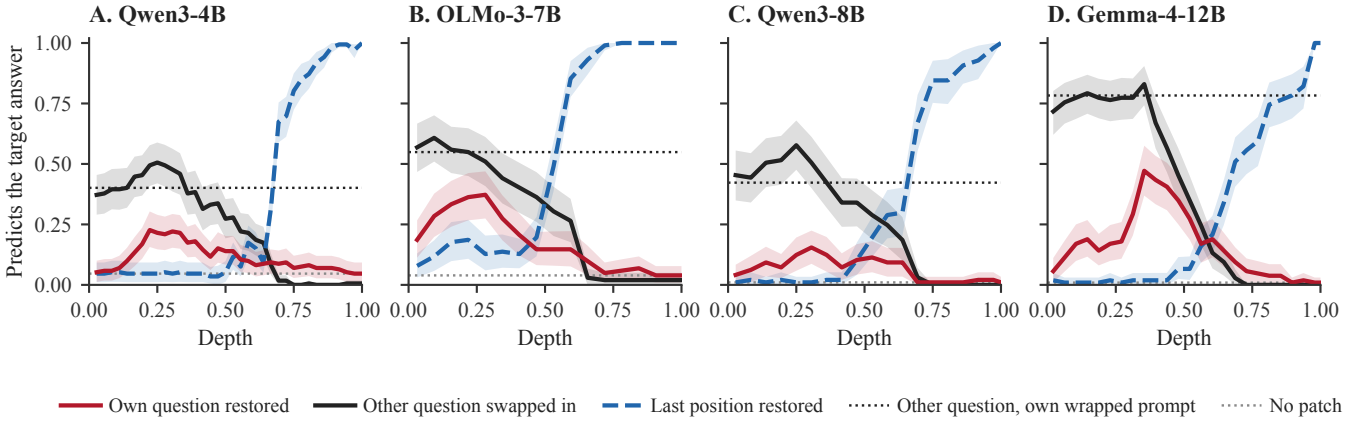


Figure 4: Patching failed wrapped prompts, by the depth of the patch. Each solid or dashed line is the share of failed test prompts in which, after one patch at one layer, the model's most likely next token is the first token of the target answer. Red: the question's tokens are restored from the question's plain prompt; the target is its own answer. Black: the question's tokens are replaced by those of another question's plain prompt; the target is the other question's answer. Blue: the last position is restored from the plain prompt; the target is the question's own answer. Black dotted: how often the other question's answer is the most likely next token when that question is itself asked in the same wrapper. Grey dotted: the failed prompt without any patch. Bands are 95% intervals.

answer loses out later, at the position where the reply begins. Activation patching [14, 22] can tell these apart. It reruns a failed wrapped prompt, replaces one internal state with its value from another run, and checks the result.

For each model we patched failed prompts in which the reply lost the answer and the answer's probability fell at least threefold; 97–172 of them are test items (Appendix E). In each, at a single layer, we replaced one of three things:

1. *Own question restored*: the states at the question's tokens, taken from the plain prompt of the same question.
2. *Other question swapped in*: the states at the question's tokens, taken from the plain prompt of a different known question of the same kind and length in tokens, such as the capital of Sudan in place of the capital of Mongolia.
3. *Last position restored*: the state at the last token of the prompt, from which the reply is produced, taken from the plain prompt of the same question.

A state at a layer is the residual stream after that layer, and *depth* is the fraction of the model's layers it has passed through. After each patch we checked whether the model's most likely next token is the first token of the target answer: the failed question's own answer for patches 1 and 3, and the other question's answer for patch 2. Without a patch this happens in 1%–5% of the failed prompts.

Two parts of Figure 4 hold by construction. At the first layer, swapping in another question's states is almost the same as asking that question inside the wrapper. At the last layer, restoring the last position copies the plain prompt's output. The evidence lies between these ends, and it gives three results that hold in every model.

- *The swapped question is answered.* With another question's states in the first half of the network, the model gives the other question's answer in up to 51%–83% of the prompts. The other question, asked inside the same wrapper, gets its answer as the most likely token in 40%–

78% of cases. Up to about 0.59–0.69 of the depth, the question's tokens decide which question the model answers, and the wrapper does not stop the model from answering it from memory.

- *Restoring the question's own states helps much less.* The failed question's own plain states bring its answer back in at most 47% of prompts at any single layer, and in 18%–43% when restored at every layer at once. So the question is read about as well inside the wrapper as outside it, and its answer still loses. Logs are a partial exception: for the 19–32 log failures per model, restoring the question's states helps in up to 18%–64% of prompts, so part of what logs damage is the reading of the question itself.
- *The answer is lost at the last position, after the question arrives there.* Restoring the last position does not help early in the network. It first brings the answer back in at least half of the prompts at 0.59–0.69 of the depth, which is also where the swapped question stops having any effect. By then the question's content has moved from its own tokens to the last position, and the answer is lost after this move.

The model's probability of the full answer gives the same picture (Appendix F).

4 Related work

Format sensitivity. Changing only the format of a prompt, such as separators, casing, Markdown or JSON, can move accuracy by tens of points [6, 18], and a single prompt template gives unstable rankings of models [15]. Closest to us, Mehta [13] wrap benchmark questions in five wrapper families and find that much of the spread comes from replies that break the requested format. Liu and Chu [12] show that templates move a model's output probabilities more than the questions themselves. These studies measure accuracy. We separate replies that break the format from replies that lose the answer, use only answers the model knows, and locate where in the network the loss happens.

Distraction and long inputs. One irrelevant sentence lowers accuracy on grade-school math [19], and related but irrelevant passages mislead models [26]. Accuracy falls as inputs grow long [9, 10], even when the extra tokens are whitespace and retrieval is perfect [1], and depends on where the relevant text sits [11]. Tool-using agents degrade under noisy tool outputs [24]. We keep everything after the question fixed, vary the amount of text before it in three steps, and compare interface formats with plain prose of the same length.

Locating computation. Activation patching copies internal states from one run into another to find which states carry a behaviour [14, 22, 23]; its results depend on the metric and on the source of the copied states [7, 27]. In factual recall, attention moves information from the subject's tokens to the last position in the middle layers [4]. With misleading demonstrations, answers diverge at a critical layer [5]. Models also hold knowledge they do not state [2, 16]. We use patching with a matched control: the same states from a different question.

5 Limitations

- *Synthetic wrappers.* The wrappers imitate real interfaces but are generated from templates, and their content is bland office news. Real tool outputs, pages and logs carry content related to the task, which may pull harder or less hard.
- *One layout.* The question is followed by about 500 tokens of wrapper and one fixed instruction. We chose this layout because a first one, with about 64 tokens after the question, lost too few answers in Qwen3-4B; with it, logs hid half as many answers (Appendix C). Other instructions, or a question placed at the start, may give different rates.
- *Short answers.* The questions have short, single answers that the models know well. Longer answers, reasoning, and models with reasoning turned on are not tested. The largest model has 12 billion parameters.
- *Patching measures.* Patching copies states from a prompt with a different context and at different absolute positions. Our main measure looks only at whether the answer's first token is the most likely next token; answers that share a first token, such as some numbers, are not told apart. The log-probability of the full answer gives the same picture (Appendix F). Only Qwen3-4B was patched at every layer, and the log failures per model are few.
- *Sorting by rules.* Lost answers are sorted by fixed text rules (Appendix D), which misplace some replies.

6 Conclusion

A question placed inside a wrapper often loses an answer the model knows. Logs do this most, by turning the reply into a description of the log; the other wrappers mostly take answers the model holds weakly. Inside the network, the question itself gets through: the model reads whatever question sits in the question's tokens and can answer it from memory. The answer is lost later, at the position where the reply begins, after the question's content has arrived there. This suggests that making models robust to wrappers is less about how they read a question and more about how they decide what to reply. Whether the same holds for longer answers, reasoning models and real tool outputs remains open.

Code and data. Code, prompts, wrappers and all derived results: `https://github.com/gut-puncture/interface-formatting-study`.

References

[1] Yufeng Du, Minyang Tian, Srikanth Ronanki, Subendhu Rongali, Sravan Bodapati, Aram Galstyan, Azton Wells, Roy Schwartz, Eliu A. Huerta, and Hao Peng. Context length alone

hurts LLM performance despite perfect retrieval. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025. URL https://arxiv.org/abs/2510.05381.

[2] Zorik Gekhman, Eyal Ben David, Hadas Orgad, Eran Ofek, Yonatan Belinkov, Idan Szpektor, Jonathan Herzig, and Roi Reichart. Inside-out: Hidden factual knowledge in LLMs. arXiv:2503.15299, 2025. URL https://arxiv.org/abs/2503.15299.

[3] Gemma Team. Gemma 4 technical report. arXiv:2607.02770, 2026. URL https://arxiv.org/abs/2607.02770.

[4] Mor Geva, Jasmijn Bastings, Katja Filippova, and Amir Globerson. Dissecting recall of factual associations in auto-regressive language models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. URL https://arxiv.org/abs/2304.14767.

[5] Danny Halawi, Jean-Stanislas Denain, and Jacob Steinhardt. Overthinking the truth: Understanding how language models process false demonstrations. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2307.09476.

[6] Jia He, Mukund Rungta, David Koleczek, Arshdeep Sekhon, Franklin X. Wang, and Sadid Hasan. Does prompt formatting have any impact on LLM performance? arXiv:2411.10541, 2024. URL https://arxiv.org/abs/2411.10541.

[7] Stefan Heimersheim and Neel Nanda. How to use and interpret activation patching. arXiv:2404.15255, 2024. URL https://arxiv.org/abs/2404.15255.

[8] Evan Hernandez, Arnab Sen Sharma, Tal Haklay, Kevin Meng, Martin Wattenberg, Jacob Andreas, Yonatan Belinkov, and David Bau. Linearity of relation decoding in transformer language models. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2308.09124.

[9] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In *Conference on Language Modeling (COLM)*, 2024. URL https://arxiv.org/abs/2404.06654.

[10] Mosh Levy, Alon Jacoby, and Yoav Goldberg. Same task, more tokens: the impact of input length on the reasoning performance of large language models. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL https://arxiv.org/abs/2402.14848.

[11] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 2024. URL https://arxiv.org/abs/2307.03172.

[12] Yang Liu and Chenhui Chu. Understanding the prompt sensitivity. arXiv:2604.18389, 2026. URL https://arxiv.org/abs/2604.18389.

[13] Deep Pankajbhai Mehta. Format sensitivity index: Token-controlled prompt wrapper robustness and schema compliance in LLM benchmarking. arXiv:2607.09665, 2026. URL https://arxiv.org/abs/2607.09665.

[14] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2202.05262.

[15] Moran Mizrahi, Guy Kaplan, Dan Malkin, Rotem Dror, Dafna Shahaf, and Gabriel Stanovsky. State of what art? A call for multi-prompt LLM evaluation. *Transactions of the Association for Computational Linguistics*, 2024. URL https://arxiv.org/abs/2401.00595.

[16] Hadas Orgad, Michael Toker, Zorik Gekhman, Roi Reichart, Idan Szpektor, Hadas Kotek, and Yonatan Belinkov. LLMs know more than they show: On the intrinsic representation of LLM hallucinations. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2410.02707.

[17] Qwen Team. Qwen3 technical report. arXiv:2505.09388, 2025. URL https://arxiv.org/abs/2505.09388.

[18] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.11324.

[19] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *International Conference on Machine Learning (ICML)*, 2023. URL https://arxiv.org/abs/2302.00093.

[20] Team Olmo. Olmo 3. arXiv:2512.13961, 2025. URL https://arxiv.org/abs/2512.13961.

[21] Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. Function vectors in large language models. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.15213.

[22] Jesse Vig, Sebastian Gehrmann, Yonatan Belinkov, Sharon Qian, Daniel Nevo, Simas Sakenis, Jason Huang, Yaron Singer, and Stuart Shieber. Causal mediation analysis for interpreting neural NLP: The case of gender bias. arXiv:2004.12265, 2020. URL https://arxiv.org/abs/2004.12265.

[23] Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2211.00593.

[24] Ruipeng Wang, Yuxin Chen, Yukai Wang, Chang Wu, Junfeng Fang, Xiaodong Cai, Qi Gu, Hui Su, An Zhang, Xiang Wang, Xunliang Cai, and Tat-Seng Chua. AgentNoiseBench: Benchmarking robustness of tool-using LLM agents under noisy condition. arXiv:2602.11348, 2026. URL https://arxiv.org/abs/2602.11348.

[25] Chris Wendler, Veniamin Veselovsky, Giovanni Monea, and Robert West. Do llamas work in English? On the latent language of multilingual transformers. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL https://arxiv.org/abs/2402.10588.

[26] Siye Wu, Jian Xie, Jiangjie Chen, Tinghui Zhu, Kai Zhang, and Yanghua Xiao. How easily do irrelevant inputs skew the responses of large language models? arXiv:2404.03302, 2024. URL https://arxiv.org/abs/2404.03302.

[27] Fred Zhang and Neel Nanda. Towards best practices of activation patching in language models: Metrics and methods. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2309.16042.

A Details for reproduction

Items. Facts come from the country–capital task of Todd et al. [21] and the landmark–country, company–headquarters, star–constellation and superhero–person relations of Hernandez et al. [8]; translations come from the word lists of Wendler et al. [25]; arithmetic items are generated with a fixed seed (sums and differences of two-digit numbers, products of a two-digit and a one-digit number). The development and test split is made by hashing the item's subject, number pair or English word, so items that share a subject fall on the same side.

Known items. The plain prompt is the item after "Question:", a blank line and the closing instruction, inside the model's chat template, with "Answer:" placed at the start of the model's reply. The gold token sequence is the shortest start of the greedy reply that matches an accepted answer after normalisation (case, accents, articles and punctuation). An item is known if this sequence exists, has at most eight tokens, and the model, given the plain prompt, assigns it a probability of at least 0.5. Table 2 gives the counts.

Wrappers. A seeded generator writes each wrapper from templates and a small vocabulary of office events, names and timestamps; no language model writes any wrapper text. Every wrapper is checked to contain no accepted answer of the item it wraps. Each format has 16 wrapper texts; an item uses the same text, cut to length, at all three lengths, so that everything from the question to the end is identical across lengths. The prompts in a group share all tokens before the question, which lets us compute that part once.

Layout. We first ran Qwen3-4B with about 64 tokens of wrapper after the question. The fixed rule for choosing the layout was that wrappers at the longest length must lose at least 15 points more than the noise floor on development items. This layout lost 9.1 points more, so, as planned in advance, we reran Qwen3-4B with about 512 tokens after the question, which lost 16.4 points more. That layout is used for all models and all reported numbers, except in Appendix C.

Generation. Replies are greedy, up to 24 tokens, and stop at a newline or the end-of-turn token. Models run in bfloat16 with the Hugging Face Transformers library on one NVIDIA A100 GPU (80 GB). The whole study took 6.5 GPU hours.

Model	Layers	Known	Known test	Pairs
Qwen3-4B	36	1,184 (74%)	822	172
OLMo-3-7B	32	1,147 (71%)	813	102
Qwen3-8B	36	1,280 (80%)	890	97
Gemma-4-12B	48	1,278 (80%)	895	106

Table 2: Models. Known: items the model answers correctly in the plain prompt with probability at least 0.5, out of 1,606. Pairs: failed test prompts patched (Section 3.4).

B Exact-format accuracy

A reply is exactly right if its first line, after normalisation, is an accepted answer or starts with one followed by a space. This is what a program reading the reply would need. Table 3 compares it with the share of answers kept.

Wrapper	Kept	Exactly right
Tool result	90%	67%
Web page	91%	78%
Server log	77%	49%
Email thread	90%	81%
Plain prose	88%	86%
Neutral sentence	99%	99%

Table 3: Kept answers and exactly right replies, for all four models and three lengths together, test items.

C Less wrapper after the question

Table 4 compares, for Qwen3-4B, the share of known answers kept with about 64 and about 512 tokens of wrapper between the question and the closing instruction.

Wrapper	64 tokens after	512 tokens after
Tool result	92%	93%
Web page	92%	92%
Server log	81%	61%
Email thread	93%	93%
Plain prose	87%	87%

Table 4: Qwen3-4B: kept answers by the amount of wrapper after the question, all three lengths together, test items.

D Sorting lost answers

We read the first line of each reply that lost the answer. It is a *claim that the answer is not there* if it matches phrases such as "provided text", "does not contain", "not mentioned", "not a real" or "does not exist". Otherwise it is a *reply about the wrapper* if it names the wrapper's content, such as logs, jobs, errors, timeouts, the email or the thread. Otherwise it is a *wrong answer*. We wrote these rules after reading replies of Qwen3-4B and applied them unchanged to all models. A reply such as "Sanaa" for "Sana'a" counts as lost, so a few lost answers are spelling variants.

E Patching

Which failures. A failed prompt is eligible if its reply lost the answer, is not empty, does not begin with markup or a refusal phrase,

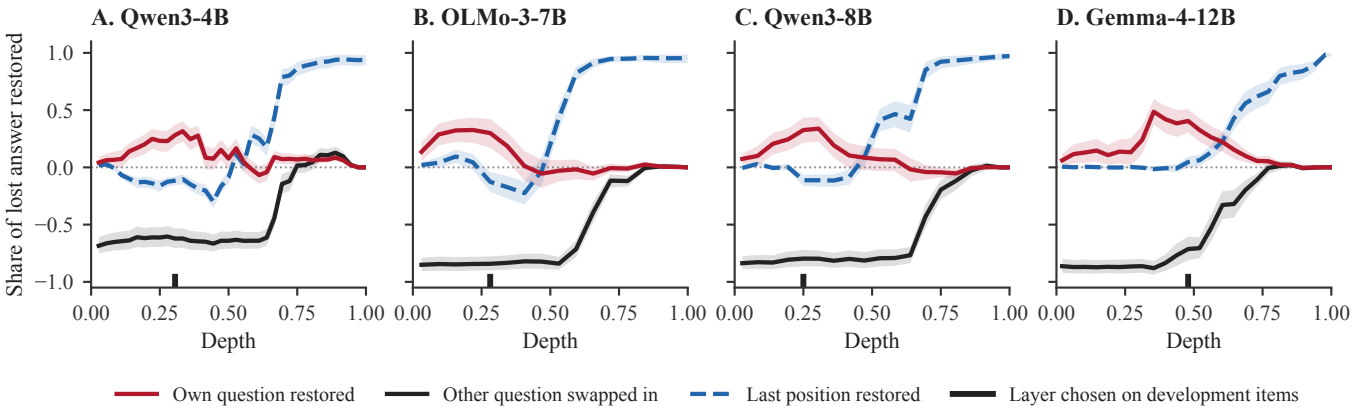


Figure 5: Share of the lost answer's log-probability restored (Equation 1), by the depth of the patch, test pairs. The tick below each axis marks ℓ^*. Bands are 95% intervals.

and, for arithmetic, contains a digit; and if the model's probability of the full answer is at least three times lower than in the plain prompt. From the eligible failures we took up to 240 for Qwen3-4B and 160 for each other model, development and test items together, cycling over item family and wrapper so that each combination is represented, with at most two prompts per item. Qwen3-4B was patched at every layer and the other models at every other layer and the last, to fit the time budget.

The other question. The swapped-in question is a different known item of the same family and wording form, with the same number of question tokens, a different subject and a different first answer token. A pair is dropped if the unpatched rerun does not reproduce the failure's log-probability within 0.05 nats, which happens through small bfloat16 differences between batch shapes; this dropped 4, 32, 32 and 25 pairs in the four models.

Positions. A patch writes each state to the token that matches its source, counted from the start of the question; the last position is matched to the last position. The plain and wrapped prompts differ in absolute position. The models encode position by rotating the attention queries and keys, not in the residual stream, so a copied state carries no absolute position of its own.

F Repair of the answer's probability

For a failed prompt, let ℓ_{plain}, ℓ_{wrap} and ℓ_{patch} be the log-probability of the full known answer in the plain prompt, in the wrapped prompt and in the wrapped prompt after a patch. The share of the lost answer restored is

$$R = \frac{\ell_{\text{patch}} - \ell_{\text{wrap}}}{\ell_{\text{plain}} - \ell_{\text{wrap}}}, \tag{1}$$

where $R = 0$ means the patch changed nothing, $R = 1$ means it restored the plain prompt's probability, and a negative R means it pushed the answer further away. We clip R to $[-1, 2]$. Figure 5 shows R by depth. It matches Figure 4. Swapping in another question pushes the answer away (R from -0.84 to -0.62) until the hand-off depth, restoring the own question recovers a part of the loss, and restoring the last position recovers it from the hand-off depth on. In the middle of the network, restoring the last position alone slightly lowers the answer's probability, before the question's content has arrived there.

We chose one layer ℓ^* on development pairs, where restoring minus swapping is largest, and tested it on test pairs (Table 5). Restoring the question's states at every layer at once does not recover more than at the best single layer.

Model	ℓ^*	Own	Other	All layers
Qwen3-4B	10	0.28 [0.19, 0.37]	-0.62 [-0.70, -0.53]	0.23 [0.11, 0.34]
OLMo-3-7B	8	0.30 [0.17, 0.42]	-0.84 [-0.90, -0.78]	0.12 [-0.03, 0.27]
Qwen3-8B	8	0.33 [0.21, 0.43]	-0.80 [-0.87, -0.72]	0.38 [0.26, 0.49]
Gemma-4-12B	22	0.41 [0.30, 0.51]	-0.71 [-0.81, -0.60]	0.34 [0.20, 0.47]

Table 5: Repair at ℓ^*, test pairs, with 95% intervals. Own: own question restored. Other: other question swapped in. All layers: own question restored at every layer at once.